

Assignment 1: Pentium M Hybrid Branch Predictor

Name: Sana Ashfaq

UH ID: 2548294

Course: COSC6385 – Computer Architecture

Repository: <https://github.com/isana25/pentium-m-branch-predictor>

1 Goal

The goal of this assignment is to simulate part of the Pentium M dynamic branch predictor inside the CBP2 (Championship Branch Prediction 2) trace-driven infrastructure, and then measure how accurately it predicts branch directions (taken / not taken) on the provided traces. The design follows the Pentium M structure reverse-engineered by Uzelac and Milenković [1]. Only the outcome (direction) predictor is implemented.

2 Design

The predictor is implemented as the `pm_predictor` class in `src/my_predictor.h`. It is a hybrid of two components.

2.1 Bimodal table

- 4096 two-bit saturating counters, indexed by `IP[11:0]` (the low 12 bits of the branch address).
- Every counter starts at 1 (weakly not taken).
- It is updated on every conditional branch.

2.2 Global predictor

- A set-associative table with **512 sets** $\times$ **2 ways**.
- Each entry stores a valid bit, a 6-bit tag, and a 2-bit saturating counter.
- A 15-bit **Global History Register (GHR)** records the outcomes of the most recent conditional branches.
- Hash: `HASH = IP[14:0] XOR GHR`
 - Set index: `HASH[14:6]` (9 bits $\rightarrow$ 512 sets)
 - Tag: `HASH[5:0]` (6 bits)
- Replacement: one LRU bit per set. On a miss, the new entry goes into an invalid way if one exists; otherwise it goes into the LRU way.
- Allocation: a new entry is created on every global miss, following the class example in Appendix B. New counters start weakly toward the actual outcome: 2 if the branch was taken, 1 if it was not.

2.3 Final prediction

- If the global table has a **tag hit** in either way, the prediction comes from that entry's counter.
- Otherwise the prediction comes from the bimodal counter.
- This matches the Pentium M rule that the global predictor overrides the bimodal predictor whenever it hits.

2.4 Update (after the real outcome is known)

1. Train the bimodal counter.
2. On a global hit, train that entry's counter and mark the other way as LRU. On a global miss, allocate an entry as described above.
3. Shift the outcome into the GHR.

Unconditional branches, calls, returns, and indirect branches are not predicted. The predictor returns “taken” for them and does not update any state.

2.5 Storage cost

Structure	Size
Bimodal table	$4096 \times 2 \text{ bits} = 8,192 \text{ bits}$
Global table	$512 \times 2 \times (1 + 6 + 2) \text{ bits} = 9,216 \text{ bits}$
LRU bits	512 bits
GHR	15 bits
Total	17,935 bits $\approx$ 2.2 KB

For comparison, the provided gshare baseline uses $32,768 \times 2 \text{ bits} = 65,536 \text{ bits}$ (8 KB), which is about $3.7\times$ more storage.

3 Simplifications (allowed by the assignment)

- **No loop predictor:** the loop branch predictor buffer (LPB) is not implemented.
- **No target prediction:** the BTB and indirect BTB are not implemented, and the target prediction is always 0.
- **No PIR:** the Path Information Register and its hash are replaced by a plain global history register XORed with the branch address, which is the same idea as gshare.
- **2 ways instead of 4** in the global predictor.

4 How to build and run

```
cd cbp2-infrastructure-v2/src
make
cd ..
./run_traces > results_pm.txt
```

The gshare comparison (extra, not required) was produced the same way with `gshare_predictor` selected in `predict.cc`, saved to `results_gshare.txt`.

On macOS, one line in `src/trace.h` was changed from `/bin/gzip` to `/usr/bin/gzip` so the traces could be decompressed.

5 Results

The metric is MPKI (direction mispredictions per 1000 instructions), as printed by `predict.cc`. Each trace is 100 million instructions. Lower is better.

Note: The assignment only requires results for the Pentium M hybrid predictor. The comparison with gshare (the predictor provided with CBP2) is an extra step I added so the results have a reference point. The “Difference” column is hybrid minus gshare; a positive number means the hybrid made more mispredictions.

Trace	gshare (8 KB)	Pentium M hybrid (2.2 KB)	Difference
164.gzip	12.473	12.617	+0.144
175.vpr	13.415	14.337	+0.922
176.gcc	11.254	15.298	+4.044
181.mcf	15.837	17.680	+1.843
186.crafty	5.837	8.464	+2.627
197.parser	10.008	10.357	+0.349
201.compress	7.831	8.199	+0.368
202.jess	1.562	1.750	+0.188
205.raytrace	2.756	3.421	+0.665
209.db	3.909	4.613	+0.704
213.javac	2.267	2.638	+0.371
222.mpegaudio	2.188	2.604	+0.416
227.mtrt	2.657	3.268	+0.611
228.jack	3.033	3.946	+0.913
252.eon	1.807	3.139	+1.332
253.perlbnk	2.554	4.700	+2.146
254.gap	3.926	4.797	+0.871
255.vortex	1.222	1.239	+0.017
256.bzip2	0.094	0.076	−0.018
300.twolf	21.489	22.046	+0.557
Average	6.305	7.259	+0.954

6 Discussion

- **Overall:** The hybrid predictor averaged 7.259 MPKI, compared with 6.305 MPKI for gshare: about 0.95 more mispredictions per 1000 instructions, while using about one quarter of the storage (2.2 KB vs 8 KB). The gap is small, which suggests the predictor is working as intended; a broken predictor would show much higher MPKI.
- **Where the hybrid did better or tied:** It beat gshare on 1 trace (bzip2) and was nearly the same on 4 (vortex +0.017, gzip +0.144, jess +0.188, parser +0.349). These programs are dominated by branches that are strongly biased toward one direction, or by a small set of

branches. The bimodal table predicts those well, and the small global table is enough for the rest.

- **Where the hybrid did worse:** The largest gaps were on gcc (+4.044), crafty (+2.627), perlbnk (+2.146) and mcf (+1.843). These programs execute many different branches whose outcomes depend on the path taken to reach them. Predicting them well needs many history patterns stored at once. The hybrid's global table holds only 1024 entries (512 sets $\times$ 2 ways), so useful entries get evicted and the predictor falls back to the bimodal table, which ignores history. Gshare's 32,768 counters can hold far more patterns.
- **Effect of the simplifications:** Using 2 ways instead of 4 halves the global table, which makes the capacity problem above worse. The full Pentium M also has a loop predictor and allocates global entries only when bimodal mispredicts, which saves room for hard branches. Adding these would likely close part of the gap with gshare.

7 Code used

- The simulation infrastructure (`cbp2-infrastructure-v2`) and the `gshare_predictor` baseline were provided by the course [2].
- No external cache code was used; the set-associative table and LRU logic were written directly in `my_predictor.h`.
- The predictor structure follows the Pentium M design reported in [1]; the reverse-engineering method behind it is described in [3].

References

- [1] Vladimir Uzelac and Aleksandar Milenković. Experiment flows and microbenchmarks for reverse engineering of branch predictor structures. In *2009 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 207–217. IEEE, 2009. Available at <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=4919652>.
- [2] Daniel A. Jiménez. Infrastructure for branch prediction competition (`cbp2`, `cbp2-infrastructure-v2`). <https://people.engr.tamu.edu/djimenez/taco/utsa-www/cs5513/competition/cbp2-infrastructure-v2/doc/index.html>, 2006. Simulation infrastructure and gshare baseline, provided by the course.
- [3] Milena Milenković, Aleksandar Milenković, and Jeffrey Kulick. Demystifying Intel branch predictors. In *Workshop on Duplicating, Deconstructing and Debunking (WDDD)*, Anchorage, Alaska, May 2002. Available at <https://pharm.ece.wisc.edu/wddd/2002/final/milenkovic.pdf>.